# 3.1 Telegram Bot完整配置

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~18k tokens，输出 ~5k tokens（本节）

如果你已经把 OpenClaw 在本地跑起来了，下一步最有成就感的事，通常就是把它接到一个你真的会用的聊天软件里。这里我建议先上 Telegram。原因很现实：它的 Bot 生态成熟，创建机器人不费劲，文档也清楚，出问题时排查路径比较短。你很容易在半小时到一小时内，把 OpenClaw 变成一个真的能在手机上聊天的助手。

这一节我们不碰太多源码，也不玩抽象概念，目标很明确：从零把 Telegram Bot 配好，接进 OpenClaw，跑通私聊和群聊。你做完这一节，应该至少能完成三件事：

1. 自己在 Telegram 里创建一个 Bot，并拿到 `bot token`
2. 把这个 Bot 写进 `openclaw.json` ，让 OpenClaw 能接收和回复消息
3. 知道私聊、群聊、Webhook、Polling 这些词分别是什么意思，出问题时从哪下手查

## 3.1.1 先把几个词说人话

第一次接 Telegram，最容易卡住的不是操作，而是术语。先把几个高频词讲明白。

**Bot**

就是 Telegram 上的机器人账号。它看起来像一个普通联系人，但背后不是人，而是程序。你后面给 OpenClaw 接的，其实不是“你的个人 Telegram 账号”，而是“你的 Telegram Bot”。

**Bot Token**

这是 Telegram 发给这个机器人的一串密钥，长得通常像这样：

```
123456789:AAHxxx_your_real_bot_token_here
```

你可以把它理解成“机器人的身份证 + 密码合体”。OpenClaw 要靠它登录 Telegram Bot API。谁拿到这个 token，谁就能控制你的机器人，所以别往公开仓库里贴，也别随手发群里。

**Webhook**

Webhook 的意思其实不复杂：Telegram 一旦收到发给你机器人的新消息，就主动“推”给你指定的一个 URL。也就是说，消息不是你去问它有没有新内容，而是它主动敲你家门。

**Polling**

Polling 刚好反过来。你的程序每隔一小段时间就去 Telegram 问一句：“有新消息吗？”如果有就拉回来处理，没有就继续等。

两者的差别可以记成一句话：

- Webhook：平台推给你
- Polling：你自己去拉

**群权限**

Telegram Bot 被拉进群以后，不一定什么都能看见、什么都能做。它能不能读消息、删消息、置顶、发链接，取决于它在群里的权限，以及群是否启用了隐私模式等设置。

## 3.1.2 Telegram 接入在 OpenClaw 里大概是什么样

从用户视角看，Telegram 接入的链路很简单：

```
你在 Telegram 发消息
-> Telegram 把消息交给 Bot API
-> OpenClaw 的 Telegram 通道收到消息
-> OpenClaw 交给 Agent 处理
-> Agent 产出回复
-> OpenClaw 再把回复发回 Telegram
```

这里最关键的一点，是 OpenClaw 不直接控制你的真人账号，而是通过“通道适配”的方式，把 Telegram 这种外部平台接到统一的内部消息流里。你暂时不用记接口名字，只要知道：OpenClaw 看待 Telegram、Discord、Slack，本质上都是“不同壳子、同一条消息流水线”。这也是为什么后面你能同时接多个通道。

## 3.1.3 第一步：找 BotFather 创建机器人

Telegram 里几乎所有 Bot 的起点，都是 `@BotFather` 。它是官方提供的机器人管理 Bot，专门负责创建、修改和发 token。

具体操作你照着做就行：

1. 打开 Telegram
2. 搜索 `@BotFather`
3. 点进聊天窗口，发送 `/start`
4. 再发送 `/newbot`
5. 它会先让你给机器人起一个显示名称，比如 `OpenClaw Study Bot`
6. 然后让你给机器人设一个用户名，这个用户名必须以 `bot` 结尾，比如 `openclaw_study_demo_bot`
7. 创建成功后，它会回你一段带 token 的消息

你会看到类似下面这种内容：

```
Done! Congratulations on your new bot.
Use this token to access the HTTP API:
123456789:AAExampleTokenxxxxxxxxxxxx
```

把这串 token 存好。最稳妥的做法是先放进密码管理器，或者临时写进本地私密笔记。别截图发朋友圈，也别贴在博客里。

**截图说明建议**

- 图 3-1：Telegram 搜索 `@BotFather` 的界面
- 图 3-2：发送 `/newbot` 后，BotFather 提示填写名称和用户名
- 图 3-3：创建成功后返回 token 的界面，记得把 token 打码

## 3.1.4 第二步：给机器人做一点基础设置

很多人创建完 Bot 就急着接 OpenClaw，其实建议顺手把几个基础项补一下。以后你会少踩很多坑。

你可以在 `@BotFather` 里继续用这些命令：

- `/setdescription` ：设置机器人简介
- `/setabouttext` ：设置“关于”说明
- `/setuserpic` ：设置头像
- `/setcommands` ：设置常用命令列表

例如你可以先配一个最小命令集：

```
start - 开始使用机器人
help - 查看帮助
reset - 重置当前对话
status - 查看当前状态
```

这一步不是必须，但用户体验会好很多。别人点开你的 Bot 时，不会觉得它像个裸奔接口。

## 3.1.5 第三步：把 token 写进 openclaw.json

OpenClaw 默认配置文件通常放在：

```
~/.openclaw/openclaw.json
```

这名字虽然叫 JSON，不过项目里一般按 JSON5 风格来写，也就是能容忍注释、尾逗号，手写起来比标准 JSON 舒服一点。

一个最小可跑的 Telegram 配置，可以先写成这样：

```
{
  gateway: {
    port: 18789,
  },

  models: {
    default: "anthropic/claude-sonnet-4-5",
  },

  agents: {
    defaults: {
      model: "anthropic/claude-sonnet-4-5",
    },
  },

  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTokenxxxxxxxxxxxx",
      dmPolicy: "pairing",
    },
  },
}
```

这里有三个字段值得停一下：

**`enabled`**

这个通道要不要启用。写成 `true` ，OpenClaw 启动时才会真的加载 Telegram 通道。

**`botToken`**

就是刚才从 BotFather 那里拿到的 token。

**`dmPolicy`**

这是私聊策略。 `pairing` 的意思可以粗暴理解成“先配对，再开放”。也就是别人第一次私聊你的 Bot 时，不会天然拥有完整访问权，而是要先经过授权流程。这个默认比直接开放更安全。

如果你只是自己本机实验，也可以暂时改成更开放的策略；但只要这个 Bot 有可能被别人碰到，我都建议先保守一点。

**截图说明建议**

- 图 3-4： `~/.openclaw/openclaw.json` 中 `channels.telegram` 配置片段

## 3.1.6 一个更实用的 Telegram 配置例子

当你不只是想“跑通”，而是想让 Telegram 更像一个日常可用入口，可以再加一点通道级设置。比如：

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTokenxxxxxxxxxxxx",

      // 私聊默认先走授权
      dmPolicy: "pairing",

      // 群里只在被@时响应，避免它像复读机一样刷屏
      mentionGating: true,

      // 允许的用户或群，没进白名单就不处理
      allowlist: {
        users: ["your_telegram_username"],
        chats: [-1001234567890],
      },

      // 自定义命令前缀示例
      commands: {
        enabled: true,
        prefix: "/",
      },
    },
  },
}
```

这里顺便解释两个新词。

**Allowlist**

以前很多人会说白名单，但现在更常见、更中性的叫法是 allowlist。意思就是：只有列表里这些人、这些群、这些目标，系统才允许通过。你可以把它理解成“可进入范围”。

**Mention Gating**

`gating` 就是“加一道门”。 `mention gating` 的意思是：在群里不是什么消息都回，只有别人明确 `@机器人` 时才回。这个功能在群聊里非常重要，不然你 Bot 可能会把整个群的日常闲聊都吃进去。

# 3.1.7 Webhook 和 Polling，到底该怎么选

很多教程会把这两个词讲得像选型大战，其实对初学者来说没那么夸张。你先按场景记：

## 场景一：本机开发、临时测试

优先用 Polling。

原因很简单：

- 不需要公网地址
- 不需要 HTTPS 证书
- 改完配置重启就能测
- 更适合你边配边试

你本地笔记本开着 OpenClaw，Bot 就能工作。这是最轻的起步姿势。

如果你想把“本地测试优先走 Polling”这件事落在配置上，可以参考下面这种写法：

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTokenxxxxxxxxxxxx",
      mode: "polling",
      dmPolicy: "pairing",
    },
  },
}
```

这里的重点不是字段名字必须和你手头版本逐字一致，而是你要知道：**Polling 模式意味着 OpenClaw 自己持续去 Telegram 拉更新。** 如果你的版本把这个模式做成自动选择，那也没关系，心智模型还是一样的。

## 场景二：长期在线、部署在服务器

优先考虑 Webhook。

原因也很直接：

- Telegram 会主动把消息推到你的服务
- 延迟通常更稳
- 不用一直轮询拉取
- 更适合 24 小时运行

但 Webhook 有前置条件：你的 OpenClaw 所在服务，得有一个 Telegram 能访问到的 HTTPS 地址。换句话说，你不能只躲在本地 `localhost` 里。

如果你后面要上服务器，Webhook 配置思路通常会长这样：

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTokenxxxxxxxxxxxx",
      mode: "webhook",
      webhook: {
        url: "https://bot.example.com/telegram/webhook",
        secret: "replace-with-a-long-random-string",
      },
    },
  },
}
```

这里的 `secret` 可以理解成额外的一道校验门。它不是给人看的，而是让你的服务在收到回调时，能更安心地确认“这请求大概率真是 Telegram 推过来的”。

所以更务实一点的建议是：

- 本地开发：先 Polling
- 有公网服务、想长期在线：再切 Webhook
- 不是非得一上来就用 Webhook，先成功比先优雅更重要

## 两者对比，一张表记住

| 方案 | 谁主动 | 适合什么场景 | 前置条件 |
| --- | --- | --- | --- |
| Polling | OpenClaw 主动拉消息 | 本机调试、快速上手 | 几乎没有 |
| Webhook | Telegram 主动推消息 | 云端部署、长期在线 | 公网 HTTPS 地址 |

如果你现在还在“第一次接通道”的阶段，我的建议很简单：先别折腾公网和证书，先用更容易成功的那一套。等你确认 Bot 能正常收发，再去换 Webhook。顺序很重要。

# 3.1.8 本地测试时，你最该先做什么

配置写完以后，不要一上来就拉群，不要一上来就测复杂命令。先做最小闭环。

## 步骤 A：启动 OpenClaw Gateway

你需要确保 Gateway 正常启动，Telegram 通道也被带起来。

## 步骤 B：在 Telegram 私聊这个 Bot

先发最简单的一句，比如：

```
你好，做个自我介绍。
```

如果用了 `pairing` 策略，第一次私聊可能先走授权流程。这不是故障，是安全机制在工作。

## 步骤 C：观察有没有这三类现象

1. 能收到消息，但不回复
2. 完全收不到消息
3. 能回复，但回复特别慢或者格式不对

这三类问题的排查方向完全不同。

# 3.1.9 Telegram 群聊接入：真正容易翻车的地方在这

私聊跑通以后，你大概率会想把机器人拉进群。这里问题就开始变多了。

先说结论：**Telegram Bot 在群里能不能正常工作，不只是 OpenClaw 配置问题，还跟 Telegram 自己的群权限和隐私模式有关。**

## 把 Bot 拉进群的基本步骤

1. 打开目标群组
2. 选择“添加成员”或“邀请成员”
3. 搜索你的 Bot 用户名

4. 添加进去
5. 如果需要，把它提升成管理员，至少给必要权限

## 建议重点检查这些权限

- 是否允许发送消息
- 是否允许读取群消息
- 是否允许回复消息
- 如果你要让它做更多管理动作，再考虑管理员权限

对于大多数“AI 助手 Bot”场景，其实不一定要管理员权限。能读、能回，通常已经够用。

## 隐私模式是什么

Telegram Bot 默认往往带一种“隐私模式”。直白点说，就是它在群里不会看见所有消息，而只会看见与它相关的内容，比如：

- 发给它的命令
- 回复它的消息
- `@` 到它的消息

这对很多 Bot 是好事，因为不容易误读整个群的聊天。但如果你希望 OpenClaw 在群里更主动地理解上下文，就要清楚这个限制。

在 `@BotFather` 里可以通过 `/setprivacy` 调整隐私模式。是否要关掉，取决于你的群使用方式：

- 小群、协作型群：可以考虑关闭，换取更完整上下文
- 大群、公开群：一般建议保持保守，避免噪音和安全问题

# 3.1.10 群里到底该怎么触发机器人

我非常不建议你让 Bot 在群里“看到什么都回”。那不是智能，是事故预备役。

比较稳的做法通常是这几种：

## 方式一：只响应 @提及

最安全，也最清楚。谁想用它，谁就 `@botname` 。

例如：

```
@openclaw_study_demo_bot 帮我总结一下刚才大家定的三件事
```

## 方式二：只响应命令前缀

比如只有 `/ask` 、 `/todo` 、 `/reset` 这种命令它才处理。

## 方式三：限定特定群组

比如它只在“研发讨论群”和“值班群”工作，别的群即使拉进去了也不响应。这时候 allowlist 特别好用。

一个偏实战的群聊配置，可以写成这样：

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTokenxxxxxxxxxxxx",
      dmPolicy: "pairing",
      mentionGating: true,
      commandGating: true,
      allowlist: {
        chats: [-1001234567890, -1009876543210],
      },
    },
  },
}
```

这里的思路是：先把门收窄，等你确认行为稳定，再一点点放开。不要反过来。

# 3.1.11 一个完整的测试流程

下面这套流程，我建议你照着测一次。它比“发一句 hello 看回不回”更接近真实使用。

## 测试 1：私聊单轮对话

给 Bot 发：

```
帮我列一个今天晚上两小时的复习计划。
```

看它是否能正常回复，回复是否像 AI 助手，而不是一段空白、报错，或者完全没回。

## 测试 2：私聊连续上下文

紧接着再发：

```
把刚才的计划压缩成 30 分钟版本。
```

如果它能理解“刚才的计划”指的是什么，说明基本上下文链路是通的。

## 测试 3：群聊 @ 提及

在群里发：

```
@openclaw_study_demo_bot 帮我总结这十分钟讨论的重点。
```

如果它只在被 `@` 时回应，说明 gating 策略生效了。

## 测试 4：非 allowlist 对象

如果你配置了 allowlist，可以用不在名单里的账号或群试一下。理想情况不是“也能用”，而是“它不处理”。安全配置能拦住，才算配对。

**截图说明建议**

- 图 3-5：Telegram 私聊中机器人正常回复的界面
- 图 3-6：群聊里通过 `@机器人` 触发的界面
- 图 3-7：allowlist 生效前后对比示意

# 3.1.12 常见问题排查，不要第一时间怪模型

很多人一看到 Bot 没回复，第一反应是“是不是模型挂了”。其实 Telegram 接入里，更常见的问题在更前面。

## 问题一：完全没反应

优先检查：

- `botToken` 是否粘贴错了
- `channels.telegram.enabled` 是否真的是 `true`

- Gateway 是否已经启动
- 你聊天的对象是不是这个 Bot，而不是名字很像的另一个

## 问题二：私聊可以，群聊不行

优先检查：

- Bot 是否真的被拉进群
- 群里是否有权限限制
- Telegram 隐私模式是否影响它看见消息
- 是否启用了 `mentionGating`，但你没有 `@` 它
- 群 ID 是否在 allowlist 里

## 问题三：群里乱回消息

这通常不是 bug，而是你把门开太大了。检查：

- 有没有开启 `mentionGating`
- 有没有开启 `commandGating`
- allowlist 是否过宽
- 是不是把隐私模式关了，但又没有做额外限制

## 问题四：回复延迟很高

Telegram 通道只是入口，真正耗时可能在：

- 模型本身慢
- 工具调用多
- 机器性能弱
- 网络链路不稳

别把所有慢都归到 Telegram 头上。

# 3.1.13 安全上要记住的几件小事

这一节虽然偏实操，但安全不能不提。尤其是 Bot 一旦接到公网，就不是“我自己电脑上玩玩”那么简单了。

- `botToken` 不要提交进 Git 仓库
- 如果怀疑泄露，立刻去 `@BotFather` 撤销并重新生成

- 新 Bot 先从私聊和小群开始测试
- 群聊默认走保守策略，别一开始就全开放
- 对外开放前，尽量打开 allowlist 或 pairing 一类门槛

如果你后面准备把 Bot 放进公开群，记得想一件事：你到底希望它是什么角色？是“被点名才回答的助手”，还是“随时插话的机器人”？前者容易控制，后者容易出事故。

## 3.1.14 从 Telegram 走到下一步

学会 Telegram 以后，你其实已经吃到了 OpenClaw 多通道体系最核心的一口：**外部平台各不相同，但接进来以后，会被收束到一套相对统一的配置和消息处理思路里。**

后面接 Discord、Slack，表面步骤不一样，底层心智模型却很像：

- 先拿到平台侧凭证
- 再写到 OpenClaw 配置
- 再决定私聊、群聊、权限、门控策略
- 最后做真实消息测试

一旦你在 Telegram 上把这套节奏跑顺了，后面通道会快很多。

## 本节小结

1. Telegram 是 OpenClaw 最适合上手的消息通道之一，因为创建 Bot 简单、调试路径短。
2. `bot token` 就是机器人登录 Telegram Bot API 的密钥，务必当成密码来保管。
3. `Polling` 更适合本地测试， `Webhook` 更适合长期在线部署；先跑通，再谈优化。
4. 群聊最容易踩坑的地方不在模型，而在群权限、隐私模式、 `@` 提及策略和 allowlist。
5. 一个稳定的 Telegram Bot，通常不是“全都能回”，而是“知道什么时候该回、什么时候别回”。